

ScratchBird Query Optimization – AI Remediation Plan

Audience: AI engineer/agent implementing fixes. Goal: apply a bounded set of changes that (1) make RLS-safe optimization provable, and (2) bring the optimizer closer to the original specification without destabilizing Alpha.

Guiding Principles

- bullet **C**orrectness and security first: RLS must be non-bypassable under all optimizer transformations.
- bullet **M**ake invariants explicit in the plan tree: rely on plan-node barriers, not tribal knowledge.
- bullet **A**dd tests before optimizations: every new transformation must have a failing test that it fixes or protects.
- bullet **P**refer small, composable steps: land one vertical slice per PR (plan node + rewrite rule + tests + explain output).
- bullet **E**nsure observability: EXPLAIN must show barrier placement and pushdown decisions.

Phase 0 – Baseline Inventory & Lockdown (1–2 PRs)

- bullet **M**ap current plan-node types and execution operators (SBLR) used for scans, filters, projections, joins, limits.
- bullet **I**dentify where policy evaluation currently occurs for SELECT (runtime hooks).
- bullet **A**dd a planner 'feature gate' config: ability to disable pushdown/rewrites/parallel plans globally for safety regression triage.
- bullet **C**reate a golden EXPLAIN snapshot test harness (plans serialized and compared in tests).

Phase 1 – Make RLS a First-Class Plan Operator (Critical)

- bullet **I**ntroduce explicit plan node: RLSFilter (or SecurityBarrierFilter).
- bullet **P**lanner must always emit: Scan(MGA-visible) → RLSFilter → UserFilter → Project → Limit.
- bullet **R**ecord in the plan which predicates are 'security predicates' vs 'user predicates'.
- bullet **E**nsure RLSFilter has access to required columns; if late materialization exists, mark required columns as 'must fetch before RLS'.
- bullet **A**dd EXPLAIN output: show RLSFilter node and policy IDs/expressions (or hashes) and whether evaluation is vectorized.

Phase 2 – Constrain Transformations Across Security Barriers

- bullet Update predicate pushdown rules: user predicates must not move below RLSFilter.
- bullet Update join reorder: RLSFilter must remain above any join that could introduce additional rows prior to policy evaluation.
- bullet Update LIMIT pushdown: forbid pushing LIMIT below RLSFilter; implement 'secure LIMIT' only if proven safe.
- bullet Update index selection: prevent index-only plans that omit policy-required columns; prefer index+heap fetch for RLS-required columns.
- bullet Add regression tests for every rule above using minimal datasets that would leak if reordered.

Phase 3 – Extended Stats (Spec Subset)

- bullet Implement the smallest useful subset: multi-column correlation support for common patterns (e.g., (a,b) joint selectivity).
- bullet Add functional dependency heuristics for equality predicates to reduce cardinality overestimation.
- bullet Persist extended stats in catalog and invalidate dependent plans on stats update.
- bullet Add targeted benchmarks and correctness tests (no negative cardinalities; sanity bounds; monotonicity checks).

Phase 4 – Plan Cache (Correctness-First)

- bullet Define cache key: normalized query + parameter types + search_path + relevant session knobs + RLS context hash.
- bullet Implement invalidation triggers: schema changes, stats updates, RLS policy changes.
- bullet Add LRU/TTL controls with safe defaults off in Alpha, on by config in Beta.
- bullet Add tests: cache hit correctness, invalidation correctness, and cross-user isolation (RLS context must be in key).

Phase 5 – Adaptive Execution (Optional, Beta-leaning)

- bullet Add runtime counters: actual rows per node, filter selectivity, hash table size, spill events.
- bullet Trigger replanning only at safe points (before join build; before large sort) and only when misestimate exceeds threshold.
- bullet Never replan in a way that moves predicates across RLSFilter.
- bullet Add fuzz/stress tests for determinism and safety.

Phase 6 – Parallel Planning (Optional, Beta-leaning)

- bullet Introduce parallel-safe operator annotation: which operators can execute in workers.
- bullet Ensure RLSFilter is evaluated per-worker with correct security context, or evaluated centrally after gather (preferred if feasible).
- bullet Cost model: account for startup cost, per-worker overhead, gather cost, skew.
- bullet Add tests ensuring identical results vs serial plans under RLS and concurrent workloads.

Deliverables & Done Criteria

Deliverable	Done when...
RLSFilter plan node	EXPLAIN shows it; tests prove no bypass under LIMIT/index/pushdown
Barrier-safe rewrite rules	Predicate pushdown/join reorder cannot cross barrier (unit tests + fuzz)
Extended stats v1	Improves cardinality on correlated predicates; catalog persistence + invalidation works
Plan cache v1	Cache keys include RLS context; invalidation on schema/stats/policy changes works
Explainability improvements	Plan output includes pushed predicates and barrier annotations

Safety Checklist (Must Not Break)

- bullet RLS correctness under: LIMIT/OFFSET, ORDER BY, DISTINCT, GROUP BY, joins, subqueries, CTEs.
- bullet No plan cache sharing across users/roles unless RLS context hash is identical.
- bullet No predicate reordering across RLS barrier; no index-only plan missing policy columns.
- bullet EXPLAIN/ANALYZE must be deterministic for the same inputs (or explicitly note nondeterminism).

Appendix – Suggested Test Matrix

- bullet RLS + index scan + LIMIT 1 (leak test)
- bullet RLS + ORDER BY + LIMIT (top-N leak test)
- bullet RLS + join (policy on one side; ensure join reorder cannot bypass)
- bullet RLS + subquery/CTE (ensure barrier stays at correct level)
- bullet RLS + parallel scan (if enabled)
- bullet Plan cache: same query different users must not share plan unless RLS context matches
- bullet Stats changes invalidate cached plans